

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 10(A)
Sorting

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

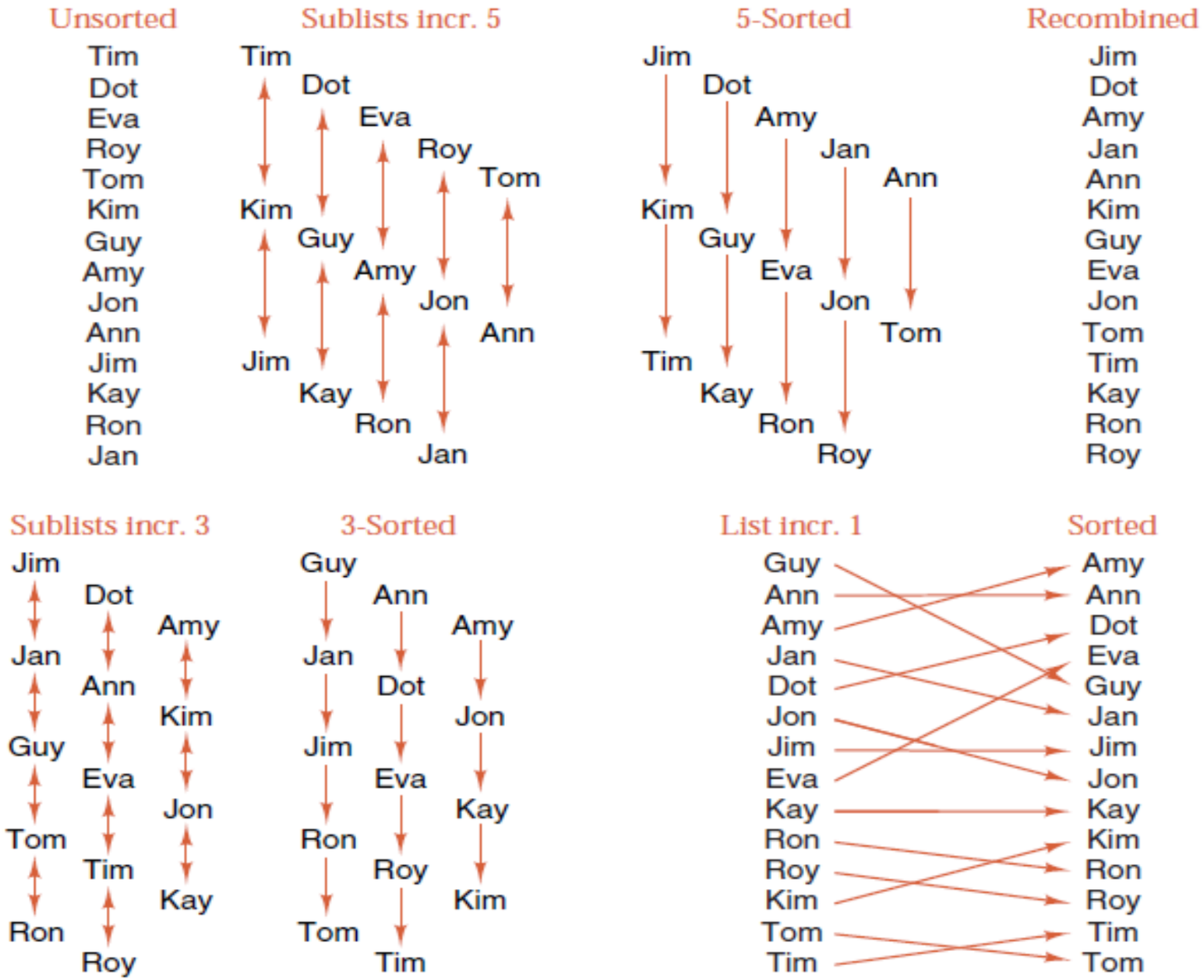
Outline

- *Introduction and Notation*
- *Insertion Sort*
- *Selection Sort*
- ----------
- Shell Sort
- Bucket Sort
- Radix Sort
- ----------
- *Divide-and-Conquer Sorting*
- *Mergesort*
- *Quicksort*
- *Heapsort*

Shell Sort (希尔排序)

- *Shell sort* was invented by *Donald Shell* in 1959. It improves upon bubble sort and insertion sort by moving out of order elements more than one position at a time.
- Insertion sort is inefficient in moving entries only one position at a time. It takes several insertion for a entry in its correct position. The reason is it only compares adjacent entries.
- How to reduce the number of movements?
- Idea: If we compare entries far apart, then it could sort the entries far apart. Finally, the increment between keys being compared is reduced to 1.

Shell Sort


Unsorted
Tim
Dot
Eva
Roy
Tom
Kim
Guy
Amy
Jon
Ann
Jim
Kay
Ron
Jan
Sublists incr. 5
Tim Dot Eva Roy Tom
Kim Guy Amy Jon Ann
Jim Kay Ron Jan
5-Sorted
Jim Dot Amy Jan Ann
Kim Guy Eva Jon Tom
Tim Kay Ron Roy
Recombined
Jim
Dot
Amy
Jan
Ann
Kim
Guy
Eva
Jon
Tom
Tim
Kay
Ron
Roy
Sublists incr. 3
Jim Dot Amy
Jan Ann Kim
Guy Eva Jon
Tom Tim Kay
Ron Roy
3-Sorted
Guy Ann Amy
Jan Dot Jon
Jim Eva Kay
Ron Roy Kim
Tom Tim
List incr. 1
Guy
Ann
Amy
Jan
Dot
Jon
Jim
Eva
Kay
Ron
Roy
Kim
Tom
Tim
Sorted
Amy
Ann
Dot
Eva
Guy
Jan
Jim
Jon
Kay
Kim
Ron
Roy
Tim
Tom

Contiguous implementation

```
template <class Record>
void Sortable_list<Record> :: shell_sort( )
/* Post:  The entries of the Sortable_list have been rearranged so that the keys in all
          the entries are sorted into nondecreasing order.
   Uses: sort_interval  */


{
   int increment,                    //   spacing of entries in sublist
       start;                        //   starting point of sublist
   increment = count;


   do {
      increment = increment/3 + 1;
      for (start = 0;  start < increment;  start++)
         sort_interval(start, increment);   //   modified insertion sort
   } while (increment > 1);
}
```

Shell Sort Example

例：关键字序列 T=(49,38,65,97,76,13,27,49*,55,04），
请写出希尔排序的具体实现过程。

r[i]	0	1	2	3	4	5	6	7	8	9	10
初态：		49	38	65	97	76	13	27	49*	55	04
第1趟 (inc=5)		13	27	49*	55	04	49	38	65	97	76
第2趟 (inc=3)		13	04	49*	38	27	49	55	65	97	76
第3趟 (inc=1)		04	13	27	38	49*	49	55	65	76	97

Shell Sort Example

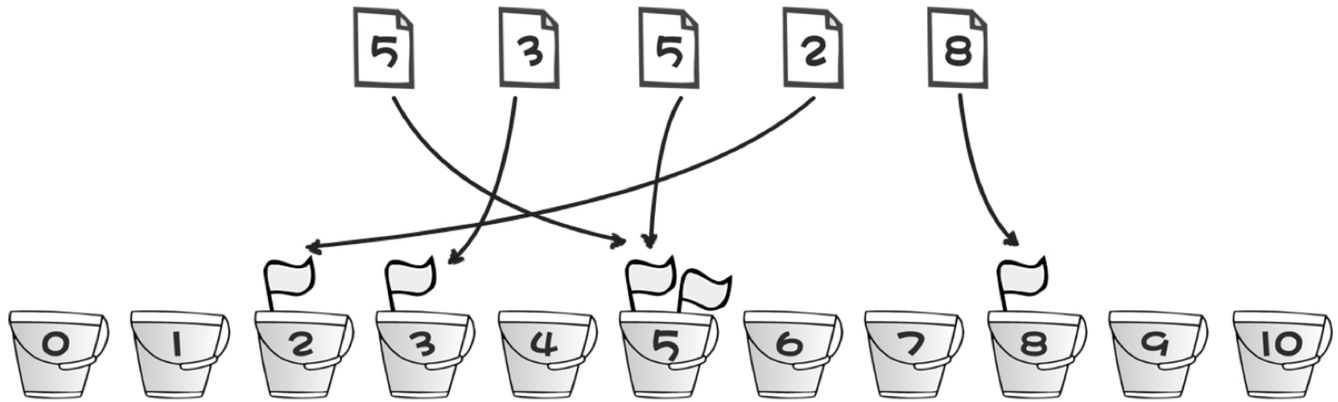
算法分析：开始时*inc* 的值较大，子序列中的对象较少，排序速度较快；随着排序进展，*inc* 值逐渐变小，子序列中对象个数逐渐变多，由于前面工作的基础，大多数对象已基本有序，所以排序速度仍然很快。

时间效率：$O(n^{4/3})$ ～$O(n^2)$——希尔增量、Hibbard增量等
空间效率：O(n)
算法的稳定性：不稳定——因为49*排序后却到了49的前面

- Exercise:
  - Use Shell Sort to sort [256，301，751，129，937，863，742，694，076，438]
  - increment = 5, 3, 1

Bucket Sort (桶排序)

- Suppose that we have N integers in the range [0, M)
- Maintain an array *count* of size M, which is initialized to zero. Thus, *count* has M cells (buckets).
- Read each integer A_i
- Increment *count*[A_i] by 1
- After all the input array is read, scan the count array, printing out a representation of sorted list.

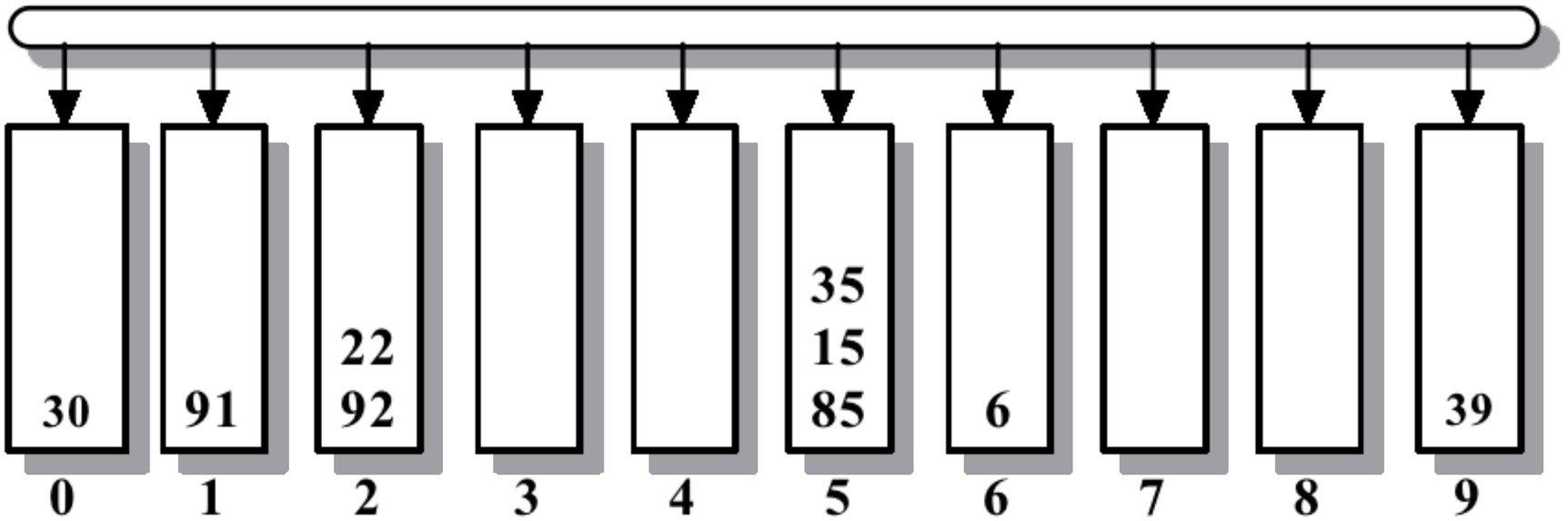


Radix Sort (基数排序)

- Radix sort is a generalization of bucket sort.
- It uses several passes of bucket sort.
- Type 1: Perform the bucket sorts by “least significant digits” (LSD)
  - First sort by digit in units place
  - Second sort by digit in tens place
  - Third sort by digit in hundreds place
  - …
- Type 2: Perform the bucket sorts by “most significant digits” (MSD)
  - https://www.geeksforgeeks.org/msd-most-significant-digit-radix-sort/

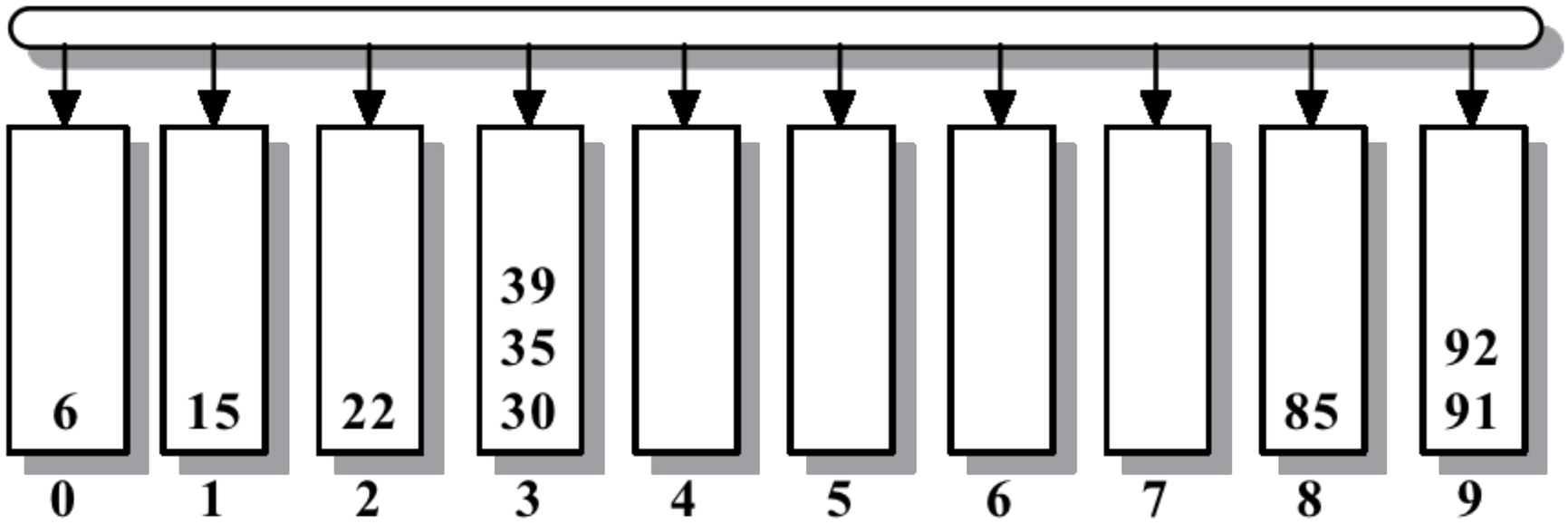
Radix Sort: Example

- Order ten 2 digit numbers in 10 bins from smallest number to largest number. Requires 2 calls to the sort algorithm.
- Initial Sequence: 91 6 85 15 92 35 30 22 39
- Pass 1: Distribute the cards into bins according to the 1's digit (10^0).
- Sequence after Pass 1: 30 91 92 22 85 15 35 6 39



Radix Sort: Example

- Sequence after Pass 1: 30 91 92 22 85 15 35 6 39
- Pass 2: Take the new sequence and distribute the cards into bins determined by the 10's digit (10^1).
- Final Sequence: 6 15 22 30 35 39 85 91 92

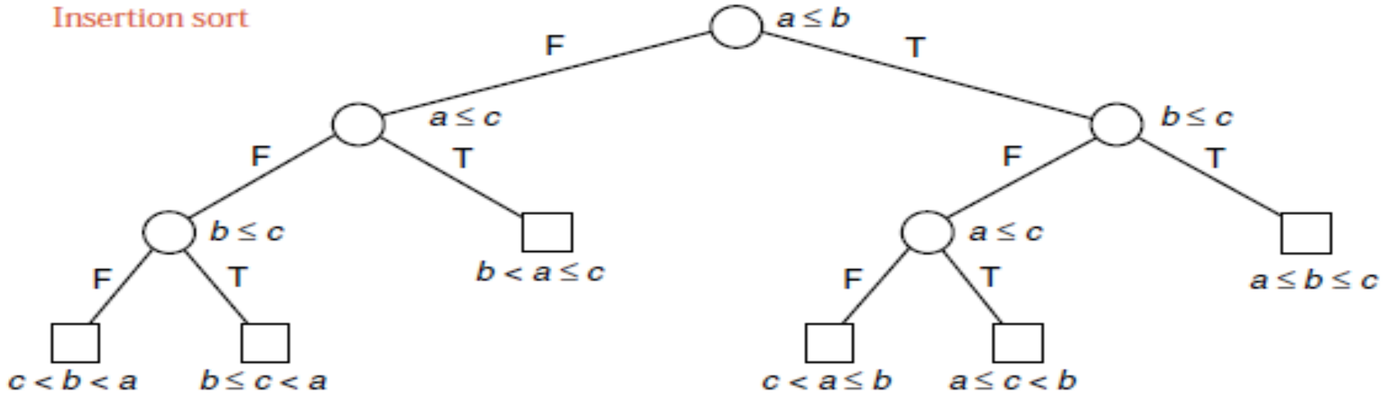


Radix Sort

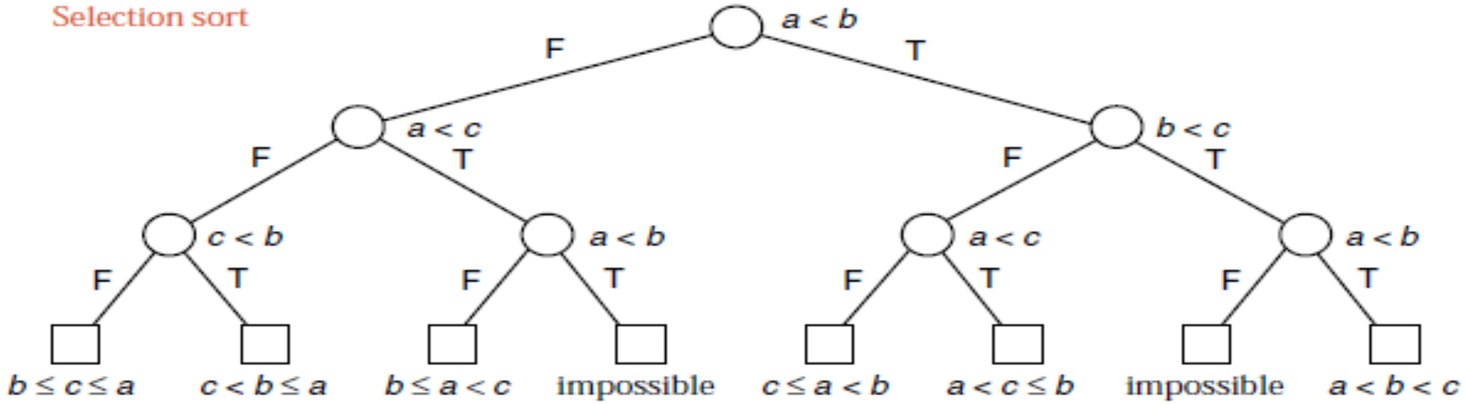
```
// 对序列ivec进行 d 位的基数排序
void radixSort(vector<int> &ivec, int d)
{
    int i, j, digit = 1;
    queue<int> digitQueue[10]; // 10个对应于位的队列
    for ( i = 0; i < d; ++i)       {
        // 将向量中的数依次进入相应位的队列中
        for (j = 0; j < ivec.size(); j++)
            digitQueue[(ivec[j] / digit) % 10].push(ivec[j]);
        // 将队列空间回收，用向量来保存排序后的序列
        j = 0;
        for (int digitVal = 0; digitVal < 10; ++digitVal)
          while (!digitQueue[digitVal].empty())
        {
              ivec[j] = digitQueue[digitVal].front();
              digitQueue[digitVal].pop();
              j++;
          }
        // 每进行一位基数排序，就将中间结果输出显示
        cout << "经过第 " << i << "次排序后得到的序列: " << endl;
        for ( j = 0; j < ivec.size(); ++j)
          cout << ivec[j] << "  ";
        cout << endl;
        // 从个位数开始，直到第d位数
        digit *= 10;
    }
}
```

Lower Bounds for Sorting

Insertion sort



Selection sort



THEOREM 8.2 Any algorithm that sorts a list of n entries by use of key comparisons must, in its worst case, perform at least $\lceil \lg n! \rceil$ comparisons of keys, and, in the average case, it must perform at least $\lg n!$ comparisons of keys.

Approximation for factorials

- Stirling's approximation

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1+\Theta\left(\frac{1}{n}\right)\right)$$

$$n! = o(n^n)$$

$$n! = \omega(2^n)$$

$$\log(n!) = \Theta(n \log n)$$

Thank you!

